Memory Efficient Doubly Linked List

–Deletion, Traversal, Destroy Operation Understanding

1. *Deletion At Beginning*

```
    Node *next = XOR(nullptr, head->npx);
```

This exhibit the identity property : $X \oplus 0 = X$.

It stores the address of 2nd Node i.e. head → npx,

, the address of second node : B.

As we know that , :

The next pointer of A is : NULL $\oplus$ Address of B

```
Node *temp = head;
if (next != nullptr)
    {
        Node *nextNext = XOR(temp, next->npx);
        next->npx = XOR(nullptr, nextNext);
    }
```

Here Temp pointer hold the address of Node A present at Head.

$$\textbf{As } next \neq NullPtr :$$

$$nextNext = XOR(prev: Address\ of Node\ A, curr \rightarrow npx: C)$$

$$Note: C \text{ is the XOR result.} \begin{bmatrix} \text{May not be address of} \\ \text{Node C i.e. Result of} \\ XOR\begin{pmatrix}\text{Address of NodeA,} \\ \text{Address of Node C}\end{pmatrix} \\ \text{The only time Node B's npx} \\ \text{will contain Address of Node A} \\ \text{When there will be only two node} \\ \text{exists.} \end{bmatrix}$$

$$Hence, nextNext$$

$$= Address\ of\ Node\ A \oplus \begin{pmatrix} Address\ of\ Node\ A \\ \oplus\ Address\ of\ Node\ C \end{pmatrix}$$

$$\textbf{And we know by property}:$$

$$(B \oplus D) \oplus D = B\ (since, D \oplus D = 0)$$

$$\therefore nextNext = Address\ of\ C.$$

```
next->npx = XOR(nullptr, nextNext);
```

$$next \rightarrow npx = XOR(NULLPTR, nextNext);$$

$$Hence\ it\ is\ very\ much\ needed\ 2nd\ Node\ (i.e.\ Node\ B) becomes\ Node\ A\ and\ hence\ next\ pointer\ of\ Node\ B\ must\ contain\ address\ of\ Node\ C.$$

$Hence, next \rightarrow npx = NULL \oplus nextNext = Address\ of\ Node\ C.$

$next \rightarrow npx = Address\ of\ Node\ C.$

$And\ , then\ we\ free\ Node\ A, the\ first\ Node\ of\ the\ list.$

$--This\ process\ continues\ \ldots$

---

2. Deletion At End

```
while (true)
 {
     next = XOR(prev, curr->npx);
     if (next == nullptr)
         break; // curr is the last node
     prev = curr;
     curr = next;
 }
```

1. $next = XOR\ (\ prev: NULL\ , curr \rightarrow npx: Address\ of\ Node\ B)$
2. $next = XOR(prev: Address\ of\ A\ , curr \rightarrow npx: C)$

$$\Longrightarrow Address\ of\ Node\ C$$

$Note:\ C\ is\ the\ XOR\ result.$ $\left[\begin{array}{c} May\ not\ be\ address\ of \\ Node\ C\ \ i.e.\ Result\ of \\ XOR\begin{pmatrix} Address\ of\ NodeA, \\ Address\ of\ Node\ C \end{pmatrix} \\ The\ only\ time\ Node\ B's\ npx \\ will\ contain\ Address\ of\ Node\ A \\ When\ there\ will\ be\ only\ two\ node \\ exists. \end{array}\right]$

$$3.\ next = XOR(prev: Address\ of\ B\ , curr \rightarrow npx: D)$$

$$\Longrightarrow Address\ of\ Node\ D$$

$$Note: D\ is\ the\ XOR\ result..\begin{bmatrix} May\ not\ be\ address\ of \\ Node\ D\ i.e.\ Result\ of \\ XOR\begin{pmatrix} Address\ of\ Node\ B, \\ Address\ of\ Node\ D \end{pmatrix} \\ The\ only\ time\ Node\ C's\ npx \\ will\ contain\ Address\ of\ Node\ B \\ when\ there\ will\ be\ only\ three\ node \\ exists.\ Similarly\ Node\ D's\ npx\ \ will \\ contain\ Address\ of\ Node\ C, when\ there\ will \\ be\ only\ four\ node\ exist. \end{bmatrix}$$

$\ldots\ldots etc.$

$Similarly\ , when\ the\ loop\ fails\ when\ we\ get\ situation\ like:$

$$next = (prev: Address\ of\ Node\ A, curr \rightarrow npx: Address\ of\ Node\ A); \begin{bmatrix} When \\ There\ is\ only \\ Two\ Node \end{bmatrix}$$

$$next = (prev: Address\ of\ Node\ B, curr \rightarrow npx: Address\ of\ Node\ B); \begin{bmatrix} When \\ There\ is\ only \\ Three\ Node \end{bmatrix}$$

$$next = (prev: Address\ of\ Node\ C, curr \rightarrow npx: Address\ of\ Node\ C); \begin{bmatrix} When \\ There\ is\ only \\ Four\ Node \end{bmatrix}$$

$\ldots\ldots etc.$

When we have more than 1 node(List with more than that of Single Node)

```
    if (prev != nullptr)
    {
        Node *prevPrev = XOR(prev->npx, curr);
        prev->npx = XOR(prevPrev, nullptr);
    }
```

Suppose we have a List of Node A, Node B, Node C and Node D. Node D will be deleted and prev pointer points to Node C and next pointer of Node C must point to previous Node B's by logic:

The next pointer of Node D is: Address of Node C $\oplus$ NULL.

prevPrev = prev → npx(Node C's npx) $\oplus$ curr: Address of D
Here, Node C's npx = Address of B $\oplus$ Address of D.

Hence , we can write:

$\Rightarrow$ prevPrev = (Address of B $\oplus$ Address of D) $\oplus$ Address of D.
By using logic: (B $\oplus$ D) $\oplus$ D = B (since , D $\oplus$ D = 0), we get:
$\Rightarrow$ prevPrev = Address of B.

$\therefore$ We know, the last node's next pointer would represent like:

The next pointer of D is C $\oplus$ NULL

Hence : prev → npx(Node C's NPX) = XOR(prevPrev, NULL)

$\Rightarrow$ ***Next Pointer of Node C = Address of B $\oplus$ NULL***

= Address of B.

And then we free the `curr` pointer variable pointed to last node.

---

3. *Deletion At Position*

```
while (curr != nullptr && i < pos)
 {
     next = XOR(prev, curr->npx);
     prev = curr;
     curr = next;
     i++;
 }
```

1. $next = XOR\ (prev: NULL\ , curr \rightarrow npx: Address\ of\ Node\ B)$

2. $next = XOR(prev: Address\ of\ A\ , curr \rightarrow npx: C)$

$\Longrightarrow Address\ of\ Node\ C$

Note: ***C is the XOR result.*** $\left[\begin{array}{c} May\ not\ be\ address\ of \\ Node\ C\ \ i.e.\ Result\ of \\ XOR\begin{pmatrix} Address\ of\ NodeA, \\ Address\ of\ Node\ C \end{pmatrix} \\ The\ only\ time\ Node\ B's\ npx \\ will\ contain\ Address\ of\ Node\ A \\ When\ there\ will\ be\ only\ two\ node \\ exists. \end{array}\right]$

$$3.\ next = XOR(prev: Address\ of\ B\ , curr \rightarrow npx: D)$$

$$\Longrightarrow Address\ of\ Node\ D$$

$$Note: D\ is\ the\ XOR\ result..\begin{bmatrix} May\ not\ be\ address\ of \\ Node\ D\ i.e.\ Result\ of \\ XOR\begin{pmatrix} Address\ of\ Node\ B, \\ Address\ of\ Node\ D \end{pmatrix} \\ The\ only\ time\ Node\ C's\ npx \\ will\ contain\ Address\ of\ Node\ B \\ when\ there\ will\ be\ only\ three\ node \\ exists.\ Similarly\ Node\ D's\ npx\ \ will \\ contain\ Address\ of\ Node\ C, when\ there\ will \\ be\ only\ four\ node\ exist. \end{bmatrix}$$

$\ldots\ldots etc.$

$Similarly\ , when\ the\ loop\ fails\ when\ we\ get\ situation\ like:$

$$next = (prev: Address\ of\ Node\ A, curr \rightarrow npx: Address\ of\ Node\ A); \begin{bmatrix} When \\ There\ is\ only \\ Two\ Node \end{bmatrix}$$

$$next = (prev: Address\ of\ Node\ B, curr \rightarrow npx: Address\ of\ Node\ B); \begin{bmatrix} When \\ There\ is\ only \\ Three\ Node \end{bmatrix}$$

$$next = (prev: Address\ of\ Node\ C, curr \rightarrow npx: Address\ of\ Node\ C); \begin{bmatrix} When \\ There\ is\ only \\ Four\ Node \end{bmatrix}$$

$\ldots\ldots etc.$

```
next = XOR(prev, curr->npx); // Decode node after curr
```

$$\underline{\textbf{\textit{If the `pos` goes from first to } n-1}}$$

$$\textbf{\textit{Eg: next} = (\textit{prev: Address of B} \oplus \textit{curr} \rightarrow \textit{npx}(\textit{npx of Node C}): \textit{D}),}$$

$$\textbf{\textit{where, D is : Address of B} \oplus \textit{Address of D, hence,}}$$

$$\Rightarrow \textbf{\textit{next} = (\textit{Address of B} \oplus (\textit{Address of B} \oplus \textit{Address of D}))}$$

$$\textbf{\textit{And , we know: by property}}\ (B \oplus D) \oplus D = B\ (\textit{since}, D \oplus D = 0)$$

$$\textbf{\textit{Hence, we get:}}\ (\textit{Address of B} \oplus (\textit{Address of B} \oplus \textit{Address of D}))$$

$$\textbf{\textit{We get next} = \textit{Address of D}.}$$

$$\underline{\textbf{\textit{At nth (last node, say Node D)}}}$$

$$\Rightarrow \textbf{\textit{next} = (\textit{prev: Address of Node C} \oplus \textit{curr} \rightarrow \textit{npx}(\textit{npx of Node D}): \textit{D}),}$$

$$\textbf{\textit{where, D is : Address of C} \oplus \textit{NULL} = \textit{Address of C, hence,}}$$

$$\Rightarrow \textbf{\textit{next} = \textit{XOR}(\textit{prev: Address of Node C} \oplus \textit{curr} \rightarrow \textit{npx: Address of C})}$$

$$\Rightarrow \textbf{\textit{next} = \textit{NULL}.}$$

```
Node *prevPrev = XOR(prev->npx, curr);
prev->npx = XOR(prevPrev, next);
```

$$When\ curr\ is\ not\ pointing\ to\ end\ of\ the\ list\ \begin{bmatrix} The\ last\ Node \\ not\ to\ be\ deleted \end{bmatrix}$$

$$Suppose\ we\ have\ a\ List\ of\ Node\ A, Node\ B, Node\ C\ and\ Node\ D.$$
$$Node\ C\ will\ be\ deleted\ and\ prev\ pointer\ points\ to\ Node\ B\ and$$
$$Node\ C\ will\ get\ deleted\ then\ , next\ pointer\ logic:$$

$$The\ next\ pointer\ of\ B\ is: Address\ of\ Node\ A\ \oplus Address\ of\ Node\ C.$$

$$Where\ as\ Curr\ points\ to\ Node\ C\ means\ , Curr\ holds\ Address\ of\ Node\ C.$$

$$Hence\ , prevPrev =$$
$$prev \rightarrow npx: (Address\ of\ Node\ A\ \oplus Address\ of\ Node\ C)$$
$$\oplus\ curr: Address\ of\ Node\ C.$$

$$= (Address\ of\ Node\ A\ \oplus Address\ of\ Node\ C) \oplus Address\ of\ Node\ C.$$

$$\Rightarrow And, as\ we\ know: (B\ \oplus D) \oplus D = B\ (since\ , D \oplus D = 0)$$

$$\therefore (Address\ of\ Node\ A\ \oplus Address\ of\ Node\ C) \oplus Address\ of\ Node\ C.$$
$$= Address\ of\ Node\ A\ .$$

$$As, Node\ C\ will\ get\ deleted\ , the\ Nodes\ will\ be\ left\ is:$$
$$Node\ A, Node\ B\ and\ Node\ D\ , hence,$$
$$NPX\ of\ Node\ B\ is: Address\ of\ A\ \oplus Address\ of\ D.$$

$$Hence,$$

$$prev \rightarrow npx = prevPrev : Address\ of\ Node\ A \oplus next: Address\ of\ D.$$

$$The\ exact\ XORing\ we\ need\ to\ update\ Node\ B.$$

<u>When we want to delete the last node</u>

$$Suppose\ we\ have\ a\ List\ of\ Node\ A, Node\ B, Node\ C\ and\ Node\ D.$$
$$Node\ D\ will\ be\ deleted\ and\ prev\ pointer\ points\ to\ Node\ C\ and$$
$$next\ pointer\ of\ Node\ C\ must\ point\ to\ previous\ Node\ B's\ by\ logic:$$

$$The\ next\ pointer\ of\ Node\ D\ is: Address\ of\ Node\ \ C \oplus NULL.$$

$$Suppose\ we\ have\ a\ List\ of\ Node\ A, Node\ B, Node\ C\ and\ Node\ D.$$
$$Node\ D\ will\ be\ deleted\ and\ prev\ pointer\ points\ to\ Node\ C\ and$$
$$next\ pointer\ of\ Node\ C\ must\ point\ to\ previous\ Node\ B's\ by\ logic:$$

$$The\ next\ pointer\ of\ Node\ D\ is: Address\ of\ Node\ \ C \oplus NULL.$$

$$prevPrev = prev \rightarrow npx(Node\ C's\ npx) \oplus curr: Address\ of\ D$$
$$Here, Node\ C's\ npx = Address\ of\ B \oplus Address\ of\ D.$$

$$Hence\ ,we\ can\ write:$$

$$\Longrightarrow prevPrev = (Address\ of\ B\ \oplus Address\ of\ D) \oplus Address\ of\ D.$$

$$By\ using\ logic: (B\ \oplus D) \oplus D = B\ (\ since\ , D \oplus D = 0), we\ get:$$

$$\Longrightarrow prevPrev = Address\ of\ B.$$

$$\therefore We\ know, the\ last\ node's\ next\ pointer\ would\ represent\ like:$$

$$The\ next\ pointer\ of\ D\ is\ C\ \oplus NULL$$

$$Hence: prev \rightarrow npx(Node\ C's\ NPX) = XOR(prevPrev, NULL)$$

$$\Rightarrow Next\ Pointer\ of\ Node\ C = Address\ of\ B\ \oplus NULL$$

$$= Address\ of\ B.$$

$$And\ then\ we\ free\ the\ `curr`\ pointer\ variable\ \ pointed\ to\ last\ node.$$

```
if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(prev, nextNext);
    }
```

$$When\ next\ \neq NULL\ \begin{pmatrix} when\ last\ node\ is\ to\ be\ Deleted, next = NULL, \\ when\ last\ node\ is\ not\ be\ Deleted, next \neq NULL \end{pmatrix}:$$

$$Suppose\ there\ is\ a\ list\ composed\ of\ Node\ A, Node\ B, Node\ C\ and\ Node\ D.$$

$$If\ Node\ C\ is\ to\ get\ deleted\ then, we\ need\ to\ update:$$

$$\rightarrow next\ pointer\ of\ Node\ B.$$

$$\rightarrow and\ , next\ pointer\ of\ Node\ D.$$

$We\ have\ already\ updated\ , \rightarrow next\ pointer\ of\ Node\ B.$

$Now, we\ have\ to, \rightarrow update\ next\ pointer\ of\ Node\ D.$

$next\ pointer\ of\ Node\ D, was\ NUll\ \oplus Address\ of\ C,$
$now, it\ will\ be: next\ pointer\ of\ Node\ D, was\ NUll\ \oplus Address\ of\ B,$

$Note: Next \rightarrow NPX\ is\ NODE\ D's\ NPX\ as\ NEXT\ pointer\ points\ to\ Node\ D.$

$nextNext = Curr: Address\ of\ C\ \oplus next \rightarrow npx: Address\ of\ C\ , and\ we\ know:$

$$By\ Property: X \oplus X = 0, we\ get:$$

$nextNext = Curr: Address\ of\ C\ \oplus next \rightarrow npx: Address\ of\ C = NULL.$

$And\ next,$

```
        next->npx = XOR(prev, nextNext);
```

$next \rightarrow npx(NPX\ of\ NODE\ D) = prev: Address\ of\ B\ \oplus nextNext$
$: NULL\ .$

$next \rightarrow npx(NPX\ of\ NODE\ D) = Address\ of\ B\ .$

$And\ after\ freeing\ curr\ i.e.\ NODE\ C\ , current\ NODE\ D\ becomes\ NODE\ C.$

List Traversal and Destroy List

```
while (curr != nullptr)
  {
      next = XOR(prev, curr->npx);
      prev = curr;
      curr = next;
  }
```

1. $next = XOR\ (\ prev{:}\ NULL\ , curr \rightarrow npx{:}\ Address\ of\ Node\ B)$

2. $next = XOR(prev{:}\ Address\ of\ A\ , curr \rightarrow npx{:}\ C)$

$$\Longrightarrow Address\ of\ Node\ C$$

$$Note{:}\ C\ is\ the\ XOR\ result. \begin{bmatrix} May\ not\ be\ address\ of \\ Node\ C\ \ i.e.\ Result\ of \\ XOR\begin{pmatrix} Address\ of\ NodeA, \\ Address\ of\ Node\ C \end{pmatrix} \\ The\ only\ time\ Node\ B's\ npx \\ will\ contain\ Address\ of\ Node\ A \\ When\ there\ will\ be\ only\ two\ node \\ exists. \end{bmatrix}$$

3. $next = XOR(prev{:}\ Address\ of\ B\ , curr \rightarrow npx{:}\ D)$

$$\Longrightarrow Address\ of\ Node\ D$$

$$Note: D\ is\ the\ XOR\ result..\begin{bmatrix} May\ not\ be\ address\ of \\ Node\ D\ i.e.\ Result\ of \\ XOR\begin{pmatrix} Address\ of\ Node\ B, \\ Address\ of\ Node\ D \end{pmatrix} \\ The\ only\ time\ Node\ C's\ npx \\ will\ contain\ Address\ of\ Node\ B \\ when\ there\ will\ be\ only\ three\ node \\ exists.\ Similarly\ Node\ D's\ npx\ \ will \\ contain\ Address\ of\ Node\ C, when\ there\ will \\ be\ only\ four\ node\ exist. \end{bmatrix}$$

$\ldots\ldots etc.$

$Similarly\ , when\ the\ loop\ fails\ when\ we\ get\ situation\ like:$

$$next = (prev: Address\ of\ Node\ A, curr \rightarrow npx: Address\ of\ Node\ A); \begin{bmatrix} When \\ There\ is\ only \\ Two\ Node \end{bmatrix}$$

$$next = (prev: Address\ of\ Node\ B, curr \rightarrow npx: Address\ of\ Node\ B); \begin{bmatrix} When \\ There\ is\ only \\ Three\ Node \end{bmatrix}$$

$$next = (prev: Address\ of\ Node\ C, curr \rightarrow npx: Address\ of\ Node\ C); \begin{bmatrix} When \\ There\ is\ only \\ Four\ Node \end{bmatrix}$$

$\ldots\ldots etc.$
